Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Compiladores e Intérpretes

Proyecto I - Compilador para código binario

Compilador para una arquitectura VLIW

Integrantes:

Meibel Elena Mora Brenes	2021128644
Dylan Maximiliano Guerrero Gonzaléz	2022016016
José García Izaguirre	2022437991
Javier Hernández Castillo	2022321746

Profesor: Jason Leitón Jiménez

Grupo: 1

Semestre: II Semestre 2026

Índice

1. Resumen ejecutivo **3**

2. Arquitectura del compilador y herramientas **3**

2.1. Fases . . . 3

2.2. Organización del repositorio . . . 4

2.3. Interfaz de línea de comandos . . . 5

3. Especificación de la arquitectura destino **5**

3.1. Green card de la ISA del proyecto VLIW . . . 5

4. Especificación del lenguaje **11**

4.1. Elementos léxicos . . . 11

4.2. Declaraciones y asignación . . . 13

4.3. Arreglos y matrices . . . 13

4.4. Funciones, ámbito y comentarios . . . 13

4.5. 4.5. Precedencia y límites . . . 14

5. Algoritmos clave **14**

5.1. Reconocimiento léxico . . . 14

5.2. Parser y construcción del AST . . . 14

5.3. Análisis semántico por dos pasadas . . . 15

5.4. Tabla de símbolos . . . 15

5.5. Memoria y direcciones . . . 15

5.6. Referencias y enlaces . . . 16

5.7. Cálculo de saltos y corrección por alcance . . . 16

5.8. Bundles, dependencias y codificación . . . 16

6. Organización de memoria y convención de llamadas **17**

6.1. Mapa general . . . 17

6.2. Funciones 18

7. Resolución de referencias, saltos y llamadas 18

7.1. Caso real: control anidado de la matriz 18

7.2. Caso real: factorial 18

7.3. Diferencia entre dirección y offset 18

8. Ejercicios de demostración 18

8.1. Validación de integración con la arquitectura 19

8.2. Ejercicio 1: factorial 19

8.3. Ejercicio 2: búsqueda en matriz 22

8.4. Ejercicio 3: funciones y puntajes 25

9. Manejo de casos especiales 28

10. Pruebas y evidencias reproducibles 29

10.1. Pruebas automatizadas aportadas por el equipo 29

10.2. Compilación detallada del Ejercicio 2 30

10.3. Evidencia mínima de código máquina 30

11. Uso, reproducción y entrega 30

12. Anexos técnicos de los tres programas 31

12.1. Ejercicio 1 32

12.2. Ejercicio 2 32

12.3. Ejercicio 3 33

1. Resumen ejecutivo

BALC es un compilador escrito en C que recibe un archivo fuente en el lenguaje BAL y produce instrucciones para una microarquitectura VLIW específica. Integra reconocimiento de tokens, análisis sintáctico con AST, análisis semántico con ámbitos, cálculo de ubicaciones de memoria, generación de ensamblador, enlace y resolución de saltos, y codificación binaria. Los formatos de salida son `.asm` (representación textual por bundles), `.bin`(contenedor BALB v1) y `.mem` (hexadecimal para memoria de instrucciones).

2. Arquitectura del compilador y herramientas

2.1. Fases

```
archivo .bal
    |
    v
Lexer (Flex) -> secuencia de tokens
    |
    v
Parser (Bison) -> AST
    |
    v
Analisis semantico -> tabla de simbolos y ambitos
    |
    v
Asignacion de memoria -> globales, pila y marcos
    |
    v
Generacion de instrucciones -> ensamblador interno
    |
    v
Enlace de funciones y resolucion de referencias
    |
    v
Layout VLIW y expansion de saltos lejanos
    |
    v
Codificacion -> .asm, .bin BALB v1 y .mem
```

2.2. Organización del repositorio

Área	Archivos relevantes	Función
Entrada / CLI	`src/main.c`, `src/cli.c`, `include/cli.h`	Opciones y orquestación.
Lexer	`src/lexer.l`, `src/lexer.c`, `src/token.c`	Tokens, ubicaciones y diagnóstico léxico.
Parser y AST	`Analisis_Sintactico/gramatica.y`, `ast.c`, `puente/lexer.c`	Gramática, construcción del árbol y enlace con el lexer.
Símbolos y semántica	`Analisis_Semantico/symbol_table.c`, `semantic.c`, `semantic_modules.c`	Tipos, ámbitos, funciones y módulos.
Memoria	`Analisis_Semantico/memory.c`	Almacenes, offsets y marcos de pila.
Backend	`src/generacion_basica.c`, `generacion_control.c`, `generacion_funciones.c`, `generacion_adaptador.c`	Emisión de código para expresiones, control y funciones.
Referencias y enlace	`src/resolucion_referencias.c`, `referencias_integracion.c`, `resolucion_saltos.c`, `enlace_funciones.c`	Direcciones y saltos.
Binario	`src/codificacion_binaria.c`, `formato_binario.c`	Verificación y escritura de `.bin` y `.mem`.
Pruebas	`tests/`	Verificaciones de etapas y de programas ejemplo.

Requisitos de construcción informados por el repositorio: compilador GCC con C11, GNU Make, Flex y Bison. En Ubuntu: `sudo apt install build-essential flex bison`; luego `make` y `make test`.

2.3. Interfaz de línea de comandos

```
./balc [-o archivo.bin] [-v] [-s] [-t] [-m] [-p] programa.bal
./balc -x [-o archivo.bin] [-v] [-p] programa.asm
```

Opción	Efecto
`-o archivo`	Define nombre de salida del archivo binario.
`-v`	Informa las etapas de compilación.
`-s`	Emite también ensamblador `.asm`.
`-t`	Muestra el AST.
`-m`	Muestra símbolos y asignación de memoria.
`-p`	Imprime bundles, direcciones y codificación.
`-x`	Codifica un archivo `.asm` directamente.
`-frontend`	Analiza entrada sin ejecutar el backend.

No debe confundirse la generación de archivos con su ejecución en el procesador.

3. Especificación de la arquitectura destino

El contrato proviene de la ISA del proyecto del curso Arquitectura de Computadores I. El tamaño de instrucción es **32 bits** y cada bundle VLIW contiene **4 slots de 32 bits**, total **128 bits = 16 bytes**, asociados a **ALU, LSU, BRU y CRIPTO**. El PC de instrucciones se expresa en bytes y las direcciones de bundles están alineadas a 16 bytes. Memoria de **64 KiB**, direccionada por bytes y little-endian. Registros generales: **32 de 32 bits**. Los inmediatos y desplazamientos con signo se interpretan en complemento a dos, según el campo correspondiente.

3.1. Green card de la ISA del proyecto VLIW

Todas las instrucciones miden 32 bits: **Tipo de operación** [31:25] (7 bits) + **ID de operación** [24:21] (4 bits) + operandos (21 bits, según el tipo). Los campos de registro usan 5 bits (x0–x31).

3.1.1. Instrucciones tipo almacenar

Mnemónico	Nombre	Descripción / ejemplo
`guardap`	Guardar palabra	`M(rf1+offset) = rf2[31:0]`. Ej.: `guardap x3, x8, 5` → si x3 = 0x20, guarda en 0x25 el contenido de x8
`guardab`	Guardar byte	`M(rf1+offset) = rf2[7:0]`. Igual que `guardap` pero solo guarda el byte menos significativo.
`cargai`	Cargar palabra	`rg = M[rf1 + offset]`, con `rg` en [15:11]. Ej.: rf1 = 1000, offset = 20 → rg = M[1020].
`cargabai`	Cargar byte	`rg = signo(M8[rf1 + offset])`, con `rg` en [15:11]; el byte se extiende con signo. Ej.: rf1 = 2000, offset = 4, M8[2004] = 0x7F → rg = 0x0000007F; si fuera 0xFF → rg = 0xFFFFFFFF.

3.1.2. Instrucciones tipo inmediato

Mnemónico	Nombre	Descripción / ejemplo
`sumai`	Suma con inmediato	`rg = rf1 + inm`. Ej.: rf1 = 10, inm = 5 → rg = 15.
`restai`	Resta con inmediato	`rg = rf1 - inm`. Ej.: rf1 = 10, inm = 3 → rg = 7.
`cizqi`	Corrimiento izquierdo inmediato	`rg = rf1 << inm`. Ej.: `00000101 << 2 = 00010100` (20).
`cderi`	Corrimiento derecho lógico inmediato	`rg = rf1 >> inm`, rellena con ceros. Ej.: `00010100 >> 2 = 00000101` (5).
`caderi`	Corrimiento aritmético derecho inmediato	`rg = rf1 >>> inm`, conserva el signo. Ej.: `11110000 >>> 2 = 11111100`.
`xori`	XOR con inmediato	`rg = rf1 XOR inm`. Ej.: `1010 XOR 1100 = 0110`.
`andi`	AND con inmediato	`rg = rf1 AND inm`. Ej.: `1010 AND 1100 = 1000`.
`ori`	OR con inmediato	`rg = rf1 OR inm`. Ej.: `1010 OR 1100 = 1110`.

3.1.3. Instrucciones tipo salto

Mnemónico	Nombre	Descripción / ejemplo
`sye`	Saltar y enlazar	`rg = PC + 16; PC = PC + offset` (offset de 16 bits con signo, en bytes desde el PC del bundle y múltiplo de 16). Ej.: PC = 1024, offset = 64 → rg = 1040, PC = 1088.
`syer`	Saltar y enlazar con registro	`rg = PC + 16; PC = (rf1 + offset) AND 0xFFFFFFF0` (offset de 11 bits con signo, en bytes). Ej.: `syer x0, ra, 0` vuelve de una subrutina.

3.1.4. Instrucciones tipo control

Mnemónico	Nombre	Descripción / ejemplo
`igualsi`	Saltar si igual	`if (rf1 == rf2) PC += inm` (inm en bytes, múltiplo de 16). Ej.: `igualsi rf1=3, rf2=4, inmediato=64` salta 4 bundles.
`igualno`	Saltar si no es igual	`if (rf1 != rf2) PC += inm`. Ej.: `igualno rf1=5, rf2=6, inmediato=96` salta 6 bundles.
`menora`	Saltar si menor	`if (rf1 < rf2) PC += inm`, con signo. Ej.: `menora rf1=7, rf2=8, inmediato=48` salta 3 bundles.
`mayoroigual`	Saltar si mayor o igual	`if (rf1 >= rf2) PC += inm`, con signo. Ej.: `mayoroigual rf1=9, rf2=10, inmediato=80` salta 5 bundles.

3.1.5. Instrucciones tipo registro

Mnemónico	Nombre	Descripción / ejemplo
`suma`	Suma	`rg = rf1 + rf2`.
`resta`	Resta	`rg = rf1 - rf2`.
`cizq`	Corrimiento izquierdo	`rg = rf1 << rf2`. Ej.: rf1 = 5, rf2 = 2 → rg = 20.

Mnemónico	Nombre	Descripción / ejemplo
`cder`	Corrimiento derecho lógico	`rg = rf1 >> rf2`, rellena con ceros. Ej.: rf1 = 20, rf2 = 2 → rg = 5.
`cader`	Corrimiento aritmético derecho	`rg = rf1 >>> rf2`, conserva el signo. Ej.: rf1 = -8, rf2 = 2 → rg = -2.
`xor`	XOR	`rg = rf1 XOR rf2`. Ej.: `1010 XOR 1100 = 0110`.
`and`	AND	`rg = rf1 AND rf2`. Ej.: `1010 AND 1100 = 1000` (8).
`or`	OR	`rg = rf1 OR rf2`. Ej.: `1010 OR 1100 = 1110` (14).
`mrq`	Menor que	`rg = (rf1 < rf2) ? 1 : 0`. Ej.: rf1 = 5, rf2 = 10 → rg = 1.
`myq`	Mayor que	`rg = (rf1 > rf2) ? 1 : 0`. Ej.: rf1 = 10, rf2 = 5 → rg = 1.

3.1.6. Instrucciones tipo cripto

Mnemónico	Nombre	Descripción / ejemplo
`fsl`	Ronda Feistel	`(L_out, R_out) = (R, L XOR F(R, K))` con `K = boveda[LK][RK]`; lee el par (r1, r1+1) y escribe (rd, rd+1). Cifrar = 4 `fsl` con RK = 0..3. Ej.: `fsl rd=6, r1=2, LK=0, RK=1`.
`fsli`	Ronda Feistel inversa	`(L_out, R_out) = (R XOR F(L, K), L)`: deshace una `fsl`. Descifrar = 4 `fsli` con RK = 3..0. Ej.: `fsli rd=6, r1=2, LK=0, RK=3`.
`ell`	Escribir llave en la bóveda	`boveda[LK][off..off+1] = (rs1, rs2)`, escribe 64 bits. Ej.: `ell LK=2, off=0, rs1=4, rs2=6` y `ell LK=2, off=2, rs1=8, rs2=10` cargan la llave completa.
`vcr`	Validar credenciales	`AUTH = (M[dir_cand] == contrasena de la boveda)`; el resultado sólo queda en ESTADO. Ej.: `vcr dir_cand=0x2000`.
`camcon`	Cambiar contraseña	`contrasena = ROL(contrasena, imm)` y `M[dir] = ROL(M[dir], imm)`; requiere `AUTH = 1`. Ej.: `camcon dir=0x0010, imm=7`.

Mnemónico	Nombre	Descripción / ejemplo
`setpwd`	Inicializar contraseña	`contrasena de la boveda = rs`; sólo con `INIT = 1`, y después `INIT = 0`. Ej.: `setpwd rs=5`.

3.1.7. Pseudoinstrucciones

Mnemónico	Nombre	Descripción / ejemplo
`nop`	No operación	Se codifica como `0x00000000`.
`mov`	Mover	`mov rg, rf1` → `suma rg, rf1, x0`.
`not`	Negación bit a bit	`not rg, rf1` → `xori rg, rf1, -1`.
`ret`	Retornar	`ret` → `syer x0, ra, 0`.

3.1.8. Codificación por tipo

Tipo de instrucción	Tipo (7 bits)	Formato	IDs
Almacenar	`1001001`	Tipo | ID | rf1 [20:16] | rf2 / rg [15:11] | offset [10:0]	guardap 0000 guardab 0001 cargai 1000 cargabai 1001
Inmediato	`1000000`	Tipo | ID | rf1 [20:16] | rg [15:11] | inm [10:0]	sumai 0000 restai 0001 cizqi 0010 cderi 0011 caderi 0100 xori 0101 andi 0110 ori 0111
Salto	`1001011`	Tipo | ID | rg [20:16] | offset [15:0]	sye 0000 syer 0001 (0010–1111 reservados)

Tipo de instrucción	Tipo (7 bits)	Formato	IDs
Control	`1000001`	Tipo | ID | rf1 [20:16] | rf2 [15:11] | inm [10:0]	igualsi 0001 igualno 0010 menora 0100 mayoroigual 1000
Registro	`1101010`	Tipo | ID | rg [20:16] | rf1 [15:11] | rf2 [10:6]	suma 1000 resta 1001 cizq 1011 cder 1111 cader 1010 xor 1110 and 1100 or 1101 mrq 0101 myq 0110
Cripto	`0000010`	Tipo | ID | LK [20:19] | RK/off [18:17] | rd [16:12] | r1 [11:7] | RSV [6:0]	fsl 0000 fsli 0001 ell 0010 vcr 0011 camcon 0100 setpwd 0101 (0110–1111 reservados)

`ell` usa: Tipo | ID | LK [20:19] | off [18:17] | rs1 [16:12] | rs2 [11:7] | RSV [6:0].
`vcr` usa: Tipo | ID | dir_cand [20:5] | RSV [4:0].
`camcon` usa: Tipo | ID | dir [20:5] | imm [4:0].
`setpwd` usa: Tipo | ID | RSV [20:5] | rs[4:0].
`syer` usa: Tipo | ID | rg [20:16] | rf1 [15:11] | offset[10:0].

3.1.9. Registros

Registro	ABI	Uso
x0	cero	Constante 0
x1	ra	Dirección de retorno (`sye`, `ret`)
x2	sp	Puntero de pila
x3	gp	Puntero global
x4–x15	t0–t11	Temporales
x16–x31	s0–s15	Preservados entre llamadas

4. Especificación del lenguaje

4.1. Elementos léxicos

El lexer y la gramática reconocen identificadores, literales enteros, booleanos, cadenas y caracteres, delimitadores `()[]{},;:`, operadores y palabras reservadas. El comentario de línea usado por los ejemplos comienza con `%%`. La sintaxis emplea una nomenclatura propia

Construcción	Token / palabra BAL	Ejemplo
Función	`create_funk`	`create_funk #declare_int# sumar(...) { ... }`
Retorno	`give`	`give total;`
Entero	`declare_int`	`n * declare_int : 5;`
Booleano	`declare_boolean`	`activo * declare_boolean;`
Texto / carácter	`declare_text` `declare_char`	Declaraciones escalares.
Sin retorno	`declare_infinite_void`	`create_funk #declare_infinite_void# main()`
Condicional	`whether` `alif` `also`	Ramas condicionales.

Construcción	Token / palabra BAL	Ejemplo
Ciclo condicional	`whale` `stop`	`whale (i < 3) { ... } stop;`
Ciclo de recorrido	`cycle`	Recorrido con inicio, límite y paso.
Suma / multiplicación	`gauss` `pitagoras`	`a gauss b` `a pitagoras b`
Resta	`neumann`	`2 neumann 1`
Potencia	`descartes`	`2 descartes 4`
Residuo	`euclides`	`10 euclides 2`

Operadores de comparación	`==`
	`=/=`
	`<`
	`>`
	`<=`
	`>=`
		Condiciones booleanas.

Operadores lógicos	`&&`
	`||`
	`~`
	`^`
		Conjunción, disyunción, negación y XOR lógico.

Importación	`bring` `aka`	`bring matematicas aka math;`

Colecciones	`declare_list`
	`size`
	`add`
	`remove`
		Listas según alcance de frontend.

Excepciones	`seek` `seize`	Análisis semántico, sin afirmar ejecución.

4.2. Declaraciones y asignación

```
n * declare_int : 5;
resultado * declare_int : 1;
resultado : resultado pitagoras n;
```

El operador `*` forma parte de la sintaxis declarativa `identificador * tipo`, mientras que `:` introduce un inicializador o una asignación. El parser construye los nodos correspondientes; el analizador semántico comprueba el tipo del inicializador y la asignación.

4.3. Arreglos y matrices

```
matriz * declare_int[2][3] : [[4, 7, 1], [9, 5, 7]];
objetivo * declare_int : matriz[0][1];
```

Se registran cantidad de dimensiones y longitudes declaradas. El analizador comprueba que el número de índices sea correcto, su tipo sea entero y los índices constantes de arreglos estáticos estén dentro de rango. Las dimensiones dinámicas pueden representarse semánticamente con longitud desconocida (`0`), pero esto **no prueba** que la generación de memoria dinámica esté soportada.

4.4. Funciones, ámbito y comentarios

```
%% Ejemplo de comentario de linea
create_funk #declare_int# sumar(a * declare_int, b * declare_int) {
    give a gauss b;
}
```

Las funciones admiten parámetros, llamadas y valores de retorno. `main` debe existir exactamente una vez en el programa principal, no aceptar argumentos y retornar `declare_infinite_void`. Los ámbitos léxicos distinguen global, función y bloques anidados; no se permiten duplicados en el mismo ámbito, mientras que el traslape entre distintos ámbitos se resuelve por búsqueda del más interno al exterior.

4.5. 4.5. Precedencia y límites

De menor a mayor precedencia:

- `||`
- `^`
- `&&`
- comparaciones
- suma/resta
- multiplicación/división/residuo
- unarios
- potencia

5. Algoritmos clave

5.1. Reconocimiento léxico

El archivo `src/lexer.l` define patrones reconocibles por Flex; las reglas asocian lexemas con tokens, preservan sus posiciones y comunican errores léxicos al controlador. Las palabras reservadas deben prevalecer sobre el reconocimiento genérico de identificadores. La interfaz `src/lexer.c` adapta el analizador generado a la API del proyecto.

Flujo: leer bytes del fuente → omitir espacios y comentarios → reconocer literales/identificadores/operadores → registrar tipo y ubicación del token → entregar token a Bison.

5.2. Parser y construcción del AST

`Analisis_Sintactico/gramatica.y` define la gramática con Bison. Sus acciones construyen nodos mediante `Analisis_Sintactico/ast.c`; la conexión con los tokens está en `puente_lexer.c`. Las relaciones padre/hijos permiten representar declaraciones, expresiones, índices, bloques, bucles, llamadas y funciones. El AST permite posteriores recorridos sin volver a analizar el texto.

Flujo: consumir tokens → aplicar producciones → construir nodos AST con hijos y posición → devolver raíz `AST_PROGRAMA` → pasar a análisis semántico.

5.3. Análisis semántico por dos pasadas

En `Analisis_Semantico/semantic.c`, `analizar_semantica_con_modulos()` ejecuta:

1. **Pasada global:** registrar firmas de funciones, importaciones y constantes globales; validar el punto de entrada `main`.
2. **Pasada por funciones:** entrar al ámbito funcional, registrar parámetros, recorrer declaraciones y sentencias, comprobar tipos y ámbitos, validar retornos y salir al ámbito global.

Las funciones `registrar`, `declarar`, `expresion`, `analizar_indice`, `bloque`, `analizar_recorrido` y `garantiza_retorno` realizan validaciones específicas. `registrar` rechaza nombres duplicados en un ámbito. `analizar_indice` comprueba dimensiones y límites constantes. `garantiza_retorno` analiza rutas condicionales, sin asumir que un ciclo asegura retorno.

5.4. Tabla de símbolos

`Analisis_Semantico/symbol_table.c` organiza símbolos por ámbitos, con búsqueda desde el ámbito activo a sus ancestros y conservación de símbolos en orden de registro. Cada entrada conserva nombre, clase, tipo, ámbito, datos de arreglos, función y atributos de ubicación necesarios para el backend.

```
resolver(nombre, ambito_actual):
    mientras ambito_actual exista:
        si el nombre existe en este ambito: devolver el simbolo
        ambito_actual = ambito_actual.padre
    informar identificador no declarado
```

5.5. Memoria y direcciones

`Analisis_Semantico/memory.c` asigna espacio global y marcos de funciones. Las variables globales usan una zona basada en `gp`, mientras los parámetros y variables locales utilizan la pila `sp`. Los tipos `declare_int` ocupan 4 bytes y booleanos/caracteres 1 byte en el esquema descrito; los arreglos estáticos reservan el producto de dimensiones por el tamaño del elemento. Los offsets se conservan en la tabla de símbolos para evitar inventarlos durante la emisión.

5.6. Referencias y enlaces

El backend registra destinos simbólicos de etiquetas y funciones. Antes de codificar, calcula el layout y sustituye nombres por direcciones u offsets reales. Las referencias inválidas o sin layout se rechazan; esta propiedad fue cubierta por las pruebas `test_referencias_integracion` y `test_resolucion_referencias`

5.7. Cálculo de saltos y corrección por alcance

El desplazamiento relativo se calcula en **bytes desde el PC del bundle que contiene el salto**:

```
offset_bytes = PC_destino - PC_salto
PC_bundle = indice_bundle * 16
```

Los saltos condicionales utilizan un inmediato con signo de 11 bits y exigen múltiplos de 16, por lo que su alcance en bytes está limitado. Los saltos `sye` admiten un desplazamiento relativo con signo de 16 bits. Cuando un salto condicional queda fuera de rango, el generador puede **invertir la condición y emitir un `sye` adicional** al destino lejano; como esto altera el layout, el enlace recalcula direcciones hasta estabilizarse.

La salida real del Ejercicio 2 reporta **8 saltos resueltos, 2 reescritos y 2 vueltas de enlace**. Ejemplo real: en `0x1080`, `sye x0,-1824` conduce a `0x0960`, puesto que `0x1080 - 1824 = 0x0960`. Su palabra BRU es `9600f8e0`.

5.8. Bundles, dependencias y codificación

La configuración emite **una instrucción efectiva seguida de dos bundles `nop`**, estrategia conservadora de separación para las latencias. Los cuatro slots respetan el orden textual `ALU | LSU | BRU | CRIPTO`; la palabra hexadecimal del paquete se presenta en el orden adecuado para carga. El `.mem` utiliza 32 dígitos hexadecimales por bundle. El `.bin` contiene encabezado BALB v1 de 48 bytes, código y datos globales; por lo tanto su tamaño **no** equivale simplemente a `bundles` $\times$ `16`.

6. Organización de memoria y convención de llamadas

6.1. Mapa general

Elemento	Valor observado / contrato
Código	Comienza en `0x0000`
`gp` inicial	`0x4000`
`sp` inicial	`0x10000`, crece hacia direcciones menores
Dirección de función `main` del Ejercicio 2	`0x0330`
Tamaño del marco `main` del Ejercicio 2	48 bytes
Registro de retorno	`ra = x1` guardado en `sp+0`

En el Ejercicio 2, el prólogo realiza `restai x2,x2,48`, por lo que el `sp` en el cuerpo queda en `0xFFD0`. La matriz se ubica a partir de `sp+4`, con seis enteros de 4 bytes (24 bytes); los demás escalares ocupan 4 bytes cada uno.

Símbolo	Ámbito	Tamaño	Offset desde `sp` durante `main`	Dirección absoluta durante `main`
`matriz`	main	24	+4	`0xFFD4`
`objetivo`	main	4	+28	`0xFFEC`
`fila`	main	4	+32	`0xFFF0`
`columna`	main	4	+36	`0xFFF4`
`fila_encontrada`	main	4	+40	`0xFFF8`
`columna_encontrada`	main	4	+44	`0xFFFC`

Las direcciones absolutas se obtienen por cálculo a partir del prólogo y **no** son una captura de memoria del procesador. Son válidas mientras se ejecuta el cuerpo de `main`; el epílogo restituye `sp`.

6.2. Funciones

El backend reserva marcos, conserva `ra`, coloca parámetros conforme a la convención implementada y retorna mediante `syer x0,x1,0`. La prueba de integración de referencias incluye llamada recursiva; esto acredita los escenarios del software evaluados, no una ejecución de RTL.

7. Resolución de referencias, saltos y llamadas

7.1. Caso real: control anidado de la matriz

En `0x08a0`, el binario muestra `igualno x4,x0,96` (`82440060`) seguido en `0x08d0` por `sye x0,2256` (`960008d0`). Esta secuencia ilustra la técnica de salto condicional cercano más salto absoluto-relativo de mayor alcance. Para el retorno al bucle exterior, el paquete en `0x1080` contiene `sye x0,-1824` (`9600f8e0`).

7.2. Caso real: factorial

La compilación registrada para factorial produce **189 bundles, una función, siete saltos y un salto reescrito**. A modo de evidencia, en `0x0570` aparece `igualsi x4,x0,192` con BRU `822400c0`, y en `0x0b10` `sye x0,-1248` con BRU `9600fb20`.

7.3. Diferencia entre dirección y offset

Las direcciones de funciones registradas en la tabla son destinos de código. Los desplazamientos de bifurcación están codificados respecto al PC del salto. Los desplazamientos de datos (`sp + n` y `gp + n`) son otra categoría: no deben mezclarse con offsets de control de flujo.

8. Ejercicios de demostración

Los tres programas fuente completos y sus listados ensambladores se incorporan en los anexos de este documento y en el paquete complementario. Los hexadecimales se presentan en los listados por bundle y de forma resumida en este capítulo.

8.1. Validación de integración con la arquitectura

Se comprobó que las imágenes de instrucciones generadas por BALC coinciden con las utilizadas por la infraestructura de ejecución de la arquitectura VLIW.

La comparación se realizó para los tres ejercicios y todas las verificaciones finalizaron satisfactoriamente.

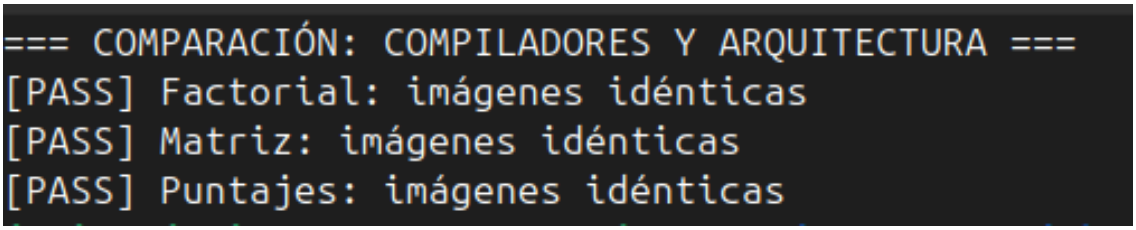


Figura 1: Comparación de imágenes de instrucciones entre Compiladores y Arquitectura.

8.2. Ejercicio 1: factorial

8.2.1. Objetivo:

calcular el factorial de 5 con variables locales, condicional y ciclo.

8.2.2. Resultado esperado:

`resultado = 120`. En el fuente, `resultado` comienza en 1 e `i` comienza en 2. En cada iteración se multiplica el acumulador por `i` y se incrementa `i`.

8.2.3. Evidencia de compilación aportada:

189 bundles, 7 saltos, 1 reescrito, archivo BAL de **3072 bytes**.

8.2.4. Evolución matemática esperada, NO ejecución observada:

Paso	`i` procesado	`resultado`
Inicial	—	1
1	2	2
2	3	6

Paso	`i` procesado	`resultado`
3	4	24
4	5	120

8.2.5. Validación de ejecución en la arquitectura VLIW

El programa factorial fue ejecutado mediante la infraestructura de simulación RTL utilizando Icarus Verilog.

La prueba de integración finalizó satisfactoriamente, confirmando la coincidencia de tres registros y tres palabras de memoria con los valores esperados.

```
FACTORIAL: n=5, resultado esperado=120
[PASS] 3 registros y 3 palabras coinciden

x14 = 0x00000078

[PASS] Imagen ejecutada identica a la de Compiladores
```

Listing 1: Validacion RTL del factorial

El registro `x14` contiene el valor hexadecimal `0x78`, equivalente a 120 en decimal.

Por tanto, la ejecución del factorial produjo el resultado esperado y la validación RTL finalizó satisfactoriamente.

```
javier@javier-HP-Laptop-15-dy2xxx:~/Documentos/GitHub/Proyecto1_Arquitectura1$ cd ~/Documentos/GitHub/Proyecto1_Arquitectura1

COMPI="../Proyecto-Parte-1-Compiladores-e-Interpretes-Definici-n-de-Sintaxis-de-Lenguaje"

mkdir -p evidencias build/validacion_compiladores
set -o pipefail

cat > build/validacion_compiladores/factorial.memory.json <<'EOF'
{"0xfff4":5,"0xfff8":120,"0xfffc":6}
EOF

cat > build/validacion_compiladores/matriz.memory.json <<'EOF'
{"0xffec":7,"0xfff0":2,"0xfff4":3,"0xfff8":0,"0xfffc":1}
EOF0":0,"sp":"0x10000","gp":"0x4000"}egistros.json <<'EOF'<'EOF'
javier@javier-HP-Laptop-15-dy2xxx:~/Documentos/GitHub/Proyecto1_Arquitectura1$ {
  echo "FACTORIAL: n=5, resultado esperado=120"
  git branch --show-current
  git rev-parse --short HEAD

  python3 tools/run_program.py \
    --input "$COMPI/examples/factorial.asm" \
    --expect build/validacion_compiladores/registros.json \
    --expect-memory build/validacion_compiladores/factorial.memory.json \
    --cycles 20000 \
    --workdir build/validacion_compiladores/factorial &&
  cmp "$COMPI/examples/factorial.mem" build/validacion_compiladores/factorial/program.mem &&
  echo "[PASS] Imagen ejecutada idéntica a la de Compiladores"
} 2>&1 | tee evidencias/02_factorial_rtl.txt
FACTORIAL: n=5, resultado esperado=120
feat/adaptador-control-compiladores
c9a3161
[PASS] 3 registros y 3 palabras coinciden
x1  = 0x000002e0
x2  = 0x00010000
x3  = 0x00004000
x5  = 0x00000005
x14 = 0x00000078
Memoria, listado, logs y registros: /home/javier/Documentos/GitHub/Proyecto1_Arquitectura1/build/validacion_compiladores/factorial
[PASS] Imagen ejecutada idéntica a la de Compiladores
```

Figura 2: Evidencia de ejecución del factorial en la arquitectura VLIW mediante Icarus Verilog.

8.2.6. Traza dinámica de ejecución

La simulación permitió observar los cambios de la variable `resultado`, almacenada en la dirección `0xFFF8`.

Durante la ejecución, su valor evolucionó de 1 a 2, 6, 24 y finalmente 120.

La repetición de la instrucción de almacenamiento en la dirección `0x0A20` evidencia las iteraciones del ciclo utilizado para calcular el factorial.

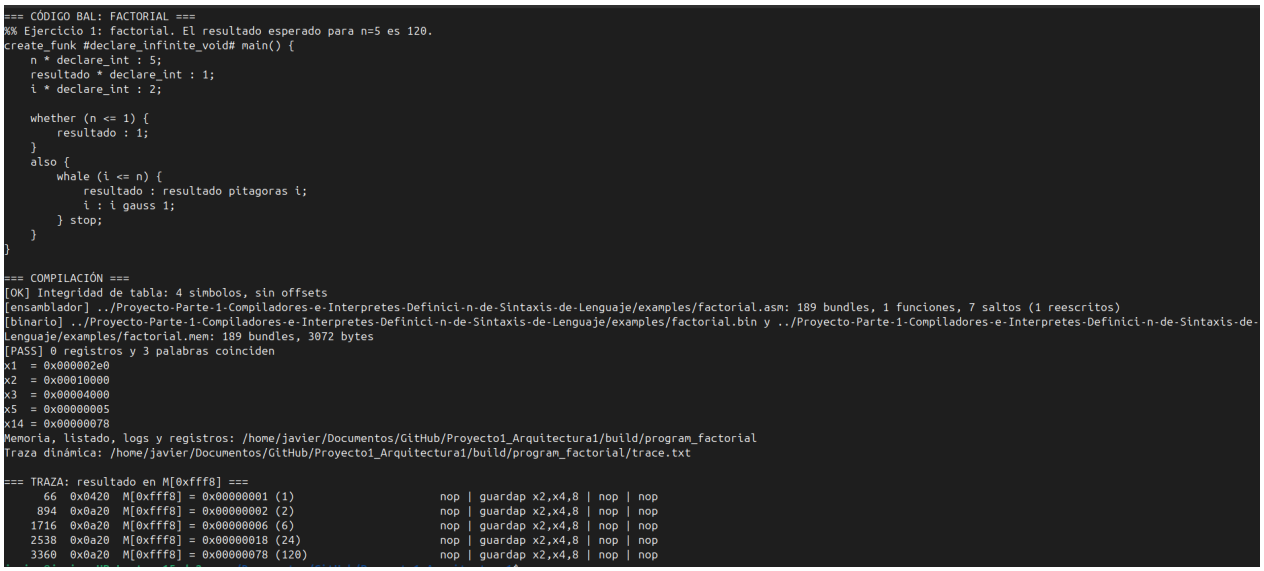


Figura 3: Traza dinámica del cálculo del factorial mediante la arquitectura VLIW.

8.3. Ejercicio 2: búsqueda en matriz

8.3.1. Objetivo:

buscar la primera aparición de 7 en `[[4,7,1],[9,5,7]]` mediante dos ciclos `whale`. El programa inicializa la posición a `(-1,-1)` y actualiza el primer hallazgo.

8.3.2. Resultado esperado:

`(fila_encontrada, columna_encontrada) = (0, 1)`.

8.3.3. Evidencia de compilación aportada:

132 tokens, AST construido, validación semántica, 7 símbolos, marco de 48 bytes, 291 bundles, 8 saltos resueltos, 2 reescritos, 2 vueltas de enlace, archivo BALB de **4704 bytes**.

8.3.4. Evolución conceptual esperada, NO ejecución observada:

Posición examinada	Valor	¿Es primer 7?	Posición almacenada
(0,0)	4	No	(-1,-1)
(0,1)	7	Sí	(0,1)

Posición examinada	Valor	¿Es primer 7?	Posición almacenada
(0,2)	1	No	(0,1)
(1,0)	9	No	(0,1)
(1,1)	5	No	(0,1)
(1,2)	7	Ya hubo un hallazgo	(0,1)

8.3.5. Validación de ejecución en la arquitectura VLIW

El programa de búsqueda en matriz fue ejecutado mediante la infraestructura de simulación RTL utilizando Icarus Verilog.

La prueba verificó la coincidencia de tres registros y cinco palabras de memoria con los valores esperados.

```
MATRIZ: posicion esperada = fila 0, columna 1
[PASS] 3 registros y 5 palabras coinciden

x5  = 0x00000002
x6  = 0x00000001

[PASS] Imagen ejecutada identica a la de Compiladores
```

Listing 2: Validacion RTL de la matriz

La validación finalizó satisfactoriamente y confirmó la coincidencia de los valores verificados con los resultados de referencia.

El resultado esperado del algoritmo es la posición $(0, 1)$, correspondiente a la primera aparición del valor buscado.

```
javier@javier-HP-Laptop-15-dy2xxx:~/Documentos/GitHub/Proyecto1_Arquitectura1$ {
  echo "MATRIZ: posición esperada = fila 0, columna 1"

  python3 tools/run_program.py \
    --input "$COMPI/examples/ejercicio2_busqueda_matriz.asm" \
    --expect build/validacion_compiladores/registros.json \
    --expect-memory build/validacion_compiladores/matriz.memory.json \
    --cycles 20000 \
    --workdir build/validacion_compiladores/matriz &&
  cmp "$COMPI/examples/ejercicio2_busqueda_matriz.mem" build/validacion_compiladores/matriz/program.mem &&
  echo "[PASS] Imagen ejecutada idéntica a la de Compiladores"
} 2>&1 | tee evidencias/03_matriz_rtl.txt
MATRIZ: posición esperada = fila 0, columna 1
[PASS] 3 registros y 5 palabras coinciden
x1  = 0x000002e0
x2  = 0x00010000
x3  = 0x00004000
x5  = 0x00000002
x6  = 0x00000001
x13 = 0x00000330
Memoria, listado, logs y registros: /home/javier/Documentos/GitHub/Proyecto1_Arquitectura1/build/validacion_compiladores/matriz
[PASS] Imagen ejecutada idéntica a la de Compiladores
```

Figura 4: Evidencia de ejecución de la búsqueda en una matriz en la arquitectura VLIW mediante Icarus Verilog.

8.3.6. Traza dinámica de ejecución

La simulación permitió verificar los cambios de las variables `fila_encontrada` y `columna_encontrada`.

Inicialmente, ambas variables fueron almacenadas con el valor -1.

Posteriormente, durante la búsqueda en la matriz, sus valores cambiaron a 0 y 1, respectivamente, confirmando que el programa identificó la primera aparición del valor buscado en la posición (0,1).

```
=== CÓDIGO BAL: BÚSQUEDA EN MATRIZ ===
%% Ejercicio 2: buscar la primera aparicion de objetivo en una matriz 2x3.
%% Al terminar: fila_encontrada, columna_encontrada = (-1, -1) si no existe.
create_funk #declare_infinite_void# main() {
    matriz * declare_int[2][3] : [[4, 7, 1], [9, 5, 7]];
    objetivo * declare_int : 7;
    fila * declare_int : 0;
    columna * declare_int : 0;
    fila_encontrada * declare_int : neumann 1;
    columna_encontrada * declare_int : neumann 1;

    whale (fila < 2) {
        columna : 0;
        whale (columna < 3) {
            whether (matriz[fila][columna] == objetivo && fila_encontrada == neumann 1) {
                fila_encontrada : fila;
                columna_encontrada : columna;
            }
            columna : columna gauss 1;
        } stop;
        fila : fila gauss 1;
    } stop;
}

=== COMPILACIÓN ===
[OK] Integridad de tabla: 7 simbolos, sin offsets
[ensamblador] ../Proyecto-Parte-1-Compiladores-e-Interpretes-Definici-n-de-Sintaxis-de-Lenguaje/examples/ejercicio2_busqueda_matriz.asm: 291
 bundles, 1 funciones, 8 saltos (2 reescritos)
[binario] ../Proyecto-Parte-1-Compiladores-e-Interpretes-Definici-n-de-Sintaxis-de-Lenguaje/examples/ejercicio2_busqueda_matriz.bin y ../Pro
yecto-Parte-1-Compiladores-e-Interpretes-Definici-n-de-Sintaxis-de-Lenguaje/examples/ejercicio2_busqueda_matriz.mem: 291 bundles, 4704 bytes
[PASS] 0 registros y 5 palabras coinciden
x1  = 0x000002e0
x2  = 0x00010000
x3  = 0x00004000
x5  = 0x00000002
x6  = 0x00000001
x13 = 0x00000330
Memoria, listado, logs y registros: /home/javier/Documentos/GitHub/Proyecto1_Arquitectura1/build/program_ejercicio2_busqueda_matriz
Traza dinámica: /home/javier/Documentos/GitHub/Proyecto1_Arquitectura1/build/program_ejercicio2_busqueda_matriz/trace.txt

=== TRAZA: fila y columna encontradas ===
     117  0x0750  M[0xfff8] = 0xffffffff (-1)                  nop | guardap x2,x4,40 | nop | nop
     126  0x07e0  M[0xfffc] = 0xffffffff (-1)                  nop | guardap x2,x4,44 | nop | nop
     339  0x0f30  M[0xfff8] = 0x00000000 (0)                   nop | guardap x2,x4,40 | nop | nop
     345  0x0f90  M[0xfffc] = 0x00000001 (1)                   nop | guardap x2,x4,44 | nop | nop
```

Figura 5: Traza dinámica de la búsqueda en una matriz bidimensional.

8.4. Ejercicio 3: funciones y puntajes

8.4.1. Objetivo:

demostrar funciones `sumar`, `puntaje` y `acumular`, parámetros, retornos, anidación de estructuras de control y llamadas encadenadas.

8.4.2. Resultado esperado:

`resultado = 24`, `vacio = 0`, `combinado = 13` y `total = 37`. El programa utiliza llamadas a funciones con parámetros, valores de retorno y ciclos para realizar los cálculos.

8.4.3. Evidencia de compilación aportada:

190 tokens, 0 errores léxicos, AST construido, validación semántica satisfactoria, 15 símbolos, 4 funciones, marco máximo de 20 bytes, 801 bundles, 6 saltos resueltos, 1 reescrito, 4 vueltas de enlace y archivo BALB de **12 864 bytes**.

8.4.4. Evolución conceptual esperada, NO ejecución observada:

Paso	Operación	Resultado
1	`acumular(5)`	24
2	`acumular(0)`	0
3	`acumular(3)`	10
4	`sumar(1,2)`	3
5	`sumar(10,3)`	13
6	`sumar(24,13)`	37

La compilación del Ejercicio 3 finalizó satisfactoriamente y generó los archivos de ensamblador y código máquina VLIW.

8.4.5. Validación de ejecución en la arquitectura VLIW

El programa de funciones y puntajes fue ejecutado mediante la infraestructura de simulación RTL utilizando Icarus Verilog.

La prueba de integración finalizó satisfactoriamente, confirmando la coincidencia de tres registros y cuatro palabras de memoria.

```
PUNTAJES: resultados esperados = 24, 0, 13 y 37
[PASS] 3 registros y 4 palabras coinciden

x4 = 0x00000025
x5 = 0x0000000d
x8 = 0x00000025
x9 = 0x0000000d

[PASS] Imagen ejecutada identica a la de Compiladores
```

Listing 3: Validacion RTL de funciones y puntajes

Los valores hexadecimales `0x25` y `0x0D` corresponden a 37 y 13 en decimal, respectivamente.

Adicionalmente, la prueba verificó cuatro palabras de memoria contra los resultados esperados.

La integración entre BALC y la arquitectura VLIW fue validada satisfactoriamente para este ejercicio.

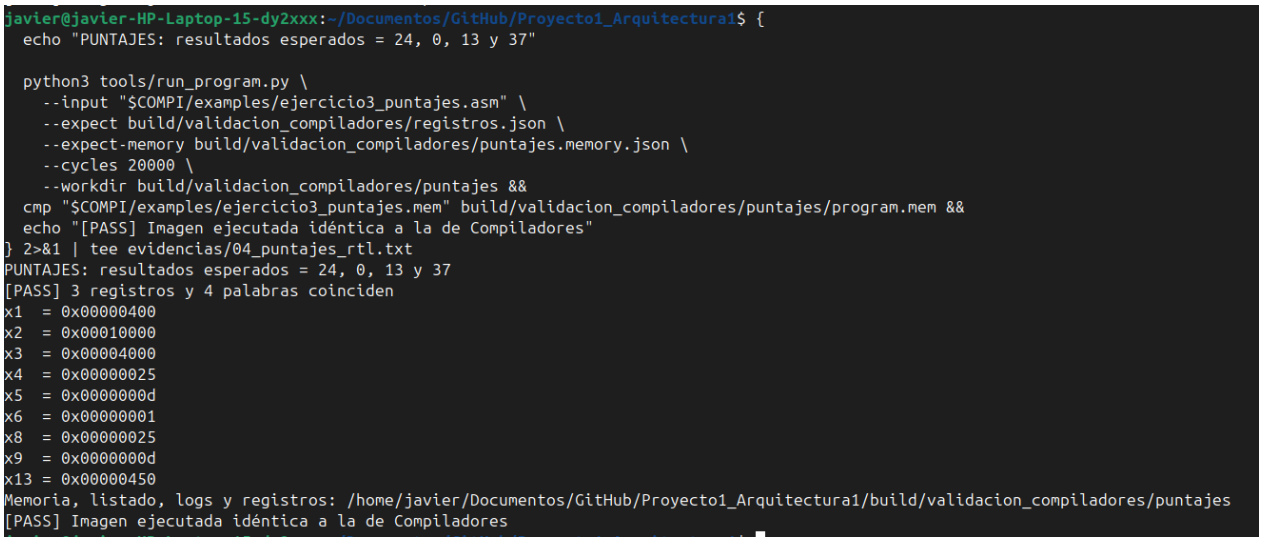


Figura 6: Evidencia de ejecución de puntajes en la arquitectura VLIW mediante Icarus Verilog.

8.4.6. Traza dinámica de ejecución

La simulación permitió verificar las escrituras en memoria correspondientes a las cuatro variables de la función `main`.

Los valores registrados fueron:

- `resultado = 24`.
- `vacio = 0`.
- `combinado = 13`.
- `total = 37`.

Estos resultados demuestran que las operaciones con funciones, parámetros y valores de retorno produjeron los resultados esperados.

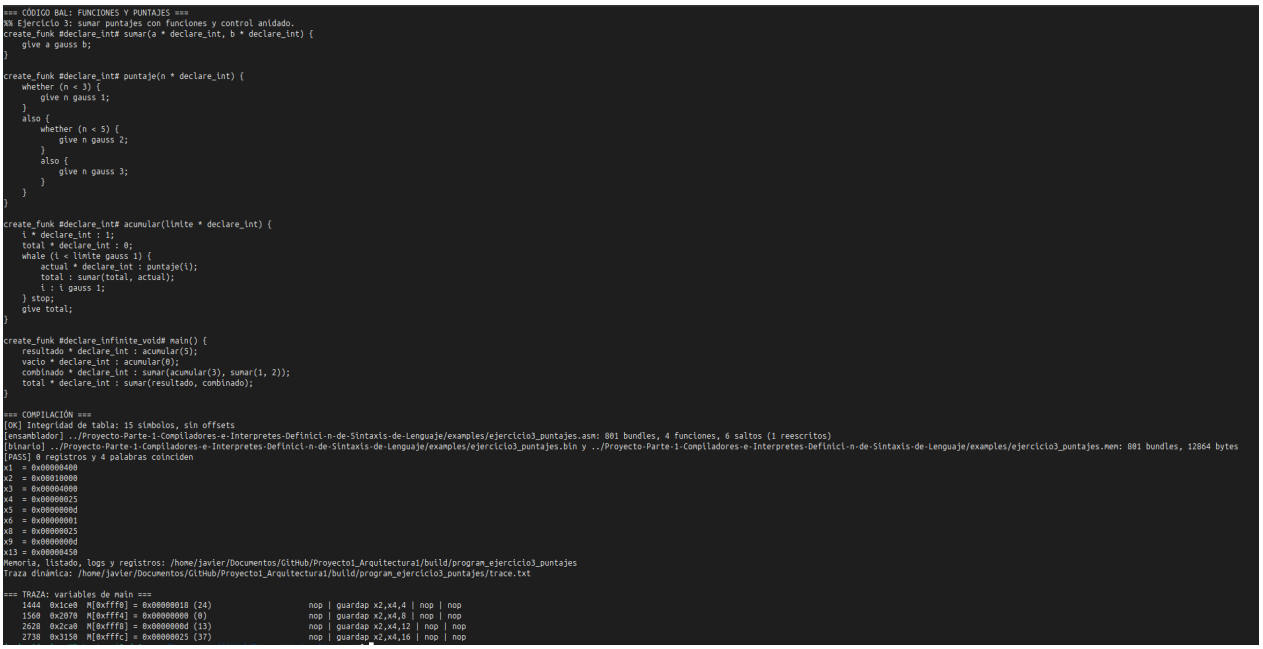


Figura 7: Traza dinámica de las funciones y el cálculo de puntajes.

9. Manejo de casos especiales

Caso	Estrategia implementada y comprobación
Condicional dentro de ciclo	AST anidado, generación de etiquetas y saltos, pruebas de layout y saltos anidados.
Salto condicional lejano	Expansión a condición invertida seguida de `sye`, con relayout hasta estabilización.
Funciones y retornos	Firmas registradas en primera pasada, parámetros y `give` validados, llamada/enlace y preservación de retorno.
Identificadores homónimos	Ámbitos de función/bloque y búsqueda desde el ámbito actual hacia los externos.
Variable global frente a local	Almacenamiento `gp` o marco `sp` con offset concreto.
Índices de matrices	Número y tipo de índices; comprobación de rango si la longitud y el índice son constantes.
Intento de modificar constante	Diagnóstico semántico incluso para destino indexado.

Caso	Estrategia implementada y comprobación
Capturas `seek/seize`	Diagnósticos semánticos y ámbitos propios; sin evidencia de runtime de excepciones.
Importaciones	Registro semántico y resolución condicionada a callback; no extrapolar a enlace multiarhivo completo.
Arreglos/listas dinámicos	Frontend y validaciones semánticas posibles; no declarar soporte total del backend sin prueba.

10. Pruebas y evidencias reproducibles

10.1. Pruebas automatizadas aportadas por el equipo

Comando	Resultado informado
`maketest-symbol-table`	Prueba exitosa.
`maketest-semantic`	OK: AST, no declarados, tipos y duplicados.
`maketest-semantic-functions`	OK: argumentos, retornos y `main`; casos de error rechazados.
`maketest-semantic-matrix-cycle`	OK: matrices, `cycle` y ámbitos.
`maketest-m-simbolos`	OK: símbolos, memoria y errores.
`maketest-auditoria-semantica`	OK: seis escenarios.
`maketest-referencias-integracion`	OK: callbacks, offsets, etiquetas, llamadas recursivas y rechazo sin layout.
`maketest-resolucion-saltos`	OK: bundles, saltos anidados y largos, funciones, arranque y arreglos.
`maketest-ejercicio2-matriz`	OK: sintaxis, semántica, `-m`, binario y rechazo de índices.

Los diagnósticos semánticos impresos en casos negativos son esperados dentro de la prueba: esta tiene éxito cuando el compilador rechaza la entrada inválida.

10.2. Compilación detallada del Ejercicio 2

```
[fase 1] lexico: 132 tokens, 0 errores
[fase 2] sintactico: AST construido
[fase 3] semantico: declaraciones, tipos, ambitos y llamadas validos
[fase 3] memoria: 1 funciones, marco maximo 48 bytes
[fase 4] ensamblador: arranque en 0x0000 llama a main; main en 0x0330
[fase 5] saltos: 8 resueltos, 2 reescritos; 2 vueltas de enlace; 291 bundles
[fase 6] binario: 291 bundles codificados y validados
[binario] ... .bin y .mem: 291 bundles, 4704 bytes
```

Interpretación: esto demuestra las etapas de compilación y generación de los archivos. No demuestra que el procesador haya ejecutado el binario.

10.3. Evidencia mínima de código máquina

```
Paquete 0138 -- PC=0x08a0
ensamblador: nop | nop | igualno x4,x0,96 | nop
hex (CRIPTO BRU LSU ALU): 00000000 82440060 00000000 00000000

Paquete 0264 -- PC=0x1080
ensamblador: nop | nop | sye x0,-1824 | nop
hex (CRIPTO BRU LSU ALU): 00000000 9600f8e0 00000000 00000000
```

11. Uso, reproducción y entrega

```
# Compilacion
make

# Ejercicios (ensamblador + binario + imagen de memoria)
./balc -v -s -m -p examples/factorial.bal
./balc -v -s -m -p examples/ejercicio2_busqueda_matriz.bal
./balc -v -s -m -p examples/ejercicio3_puntajes.bal

# Pruebas completas
make test

# Otras opciones
./balc -x -p examples/factorial.asm
```

12. Anexos técnicos de los tres programas

Los ejercicios solicitados se pueden encontrar en las siguientes rutas dentro del repositorio:

- Ejercicio 1 (Factorial): `examples/factorial.bal`
- Ejercicio 2 (Búsqueda en matriz): `examples/ejercicio2_busqueda_matriz.bal`
- Ejercicio 3 (Puntajes) : `examples/ejercicio3_puntajes.bal`

Debido a la extensión del .bal, el código ensamblador y de su representación hexadecimal, se presentan en este documento fragmentos representativos de los tres ejercicios.

Los listados completos se incluyen como archivos complementarios de la entrega, en los formatos `.asm` y `.mem`, dentro del directorio `examples/`.

12.1. Ejercicio 1

Fragmento de código fuente BAL:

```
whale (i <= n) {
    resultado : resultado pitagoras i;
    i : i gauss 1;
} stop;
```

Listing 4: Ejercicio 1: ciclo del factorial

Fragmento de ensamblador generado:

```
# PC = 0x0570
nop | nop | igualsi x4,x0,192 | nop
```

Listing 5: Ejercicio 1: salto condicional VLIW

Representación hexadecimal:

```
# PC = 0x0570
00000000822400c00000000000000000
```

Listing 6: Ejercicio 1: bundle hexadecimal

12.2. Ejercicio 2

Fragmento de código fuente BAL:

```
whale (fila < 2) {
    columna : 0;

    whale (columna < 3) {
        whether (matriz[fila][columna] == objetivo
                 && fila_encontrada == neumann 1) {
            fila_encontrada : fila;
            columna_encontrada : columna;
        }

        columna : columna gauss 1;
    } stop;

    fila : fila gauss 1;
} stop;
```

Listing 7: Ejercicio 2: busqueda en matriz

Fragmento de ensamblador generado:

```
# PC = 0x08a0
nop | nop | igualno x4,x0,96 | nop

# PC = 0x08d0
nop | nop | sye x0,2256 | nop
```

Listing 8: Ejercicio 2: resolucion de saltos

Representación hexadecimal:

```
# PC = 0x08a0
00000000824400600000000000000000

# PC = 0x08d0
00000000960008d00000000000000000
```

Listing 9: Ejercicio 2: bundles hexadecimales

12.3. Ejercicio 3

Fragmento de código fuente BAL:

```
create_funk #declare_int# sumar(
    a * declare_int,
    b * declare_int
) {
    give a gauss b;
}

create_funk #declare_int# acumular(
    limite * declare_int
) {
    i * declare_int : 1;
    total * declare_int : 0;

    whale (i < limite gauss 1) {
        actual * declare_int : puntaje(i);
        total : sumar(total, actual);
        i : i gauss 1;
    } stop;

    give total;
}
```

Listing 10: Ejercicio 3: funciones y puntajes

Fragmento de ensamblador generado:

```
# Funcion sumar, PC = 0x0570
suma x4,x4,x5 | nop | nop | nop

# Funcion acumular, PC = 0x0ea0
nop | nop | igualno x4,x0,96 | nop

# Funcion acumular, PC = 0x0ed0
nop | nop | sye x0,2400 | nop
```

Listing 11: Ejercicio 3: operaciones y saltos

Representación hexadecimal:

```
# PC = 0x0570
0000000000000000000000000d5042140

# PC = 0x0ea0
00000000824400600000000000000000

# PC = 0x0ed0
00000000960009600000000000000000
```

Listing 12: Ejercicio 3: bundles hexadecimales